



Materiały dydaktyczne

Przedmiot: Programowanie obiektowe/Pracownia programowania obiektowego

Opracował: Mirosław Miciak

Ćwiczenie: C9

Tematy: Podział programu na funkcje, Biblioteki języka programowania, Proste i złożone typy danych, Zmienne, Operacje na zmiennych: wejścia, wyjścia, arytmetyczne logiczne, Operacje na typach złożonych, Operatory arytmetyczne, przypisania, porównania, Operatory logiczne i bitowe, Operatory do obsługi łańcuchów, Instrukcja warunkowa i instrukcja wyboru, Instrukcje pętli, Wykorzystanie wybranej biblioteki matematycznej, Wykorzystanie bibliotek algorytmów, Funkcje, Klasa, obiekt, metoda, pole, Projektowanie prostych klas, Zmienne obiektowe, Zakres widoczności pól klasy, Metody klasy, Konstruktor i destruktor, Konstruktor kopiujący, Metody klasy - zakres widoczności, Prosta aplikacja obiektowa: konstruktor, destruktor, Składniki statyczne klasy, Klasy dziedziczone, Klasy zaprzyjaźnione, Dziedziczenie, Hermetyzacja, Polimorfizm, Funkcje zaprzyjaźnione z klasą, Metody do obsługi elementów statycznych klasy Klasy bazowe i pochodne, Metody wirtualne, Klasy abstrakcyjne, Szablony klas dla prostych typów liczbowych, Obsługa wyjątków, Instrukcja throw, Obsługa błędów wykonania aplikacji.

Funkcje zaprzyjaźnione z klasą

Funkcje zaprzyjaźnione (ang. friend functions) to funkcje, które nie są członkami danej klasy, ale mają dostęp do jej prywatnych i chronionych członków. W języku C# nie ma bezpośredniego odpowiednika funkcji zaprzyjaźnionych, jak w C++, ale można osiągnąć podobny efekt za pomocą metod rozszerzających (ang. extension methods) i odpowiedniego zarządzania dostępem.

Refleksja

Refleksja to mechanizm umożliwiający dynamiczne uzyskiwanie informacji o typach, metodach, polach i właściwościach w czasie wykonywania programu. Umożliwia również modyfikowanie tych elementów.

Przykład 1. Funkcje zaprzyjaźnione

```
using System;

namespace PrzykładFunkcjeZaprzyjaznione
{
    // Klasa z prywatnymi członkami
    public class MojaKlasa
    {
        private int prywatnePole = 42;

        // Prywatna metoda
        private void PrywatnaMetoda()
        {
            Console.WriteLine("Wywołano prywatną metodę");
        }

        // Metoda do wyświetlania wartości prywatnego pola
        public void PokazPrywatnePole()
        {
            Console.WriteLine("Prywatne Pole: " + prywatnePole);
        }
    }
}
```

```

// Metoda rozszerzająca, która działa jak funkcja zaprzyjaźniona
public static class MojaKlasaRozszerzenia
{
    public static void ZmienPrywatnePole(this MojaKlasa obj, int nowaWartosc)
    {
        // Użycie refleksji do modyfikacji prywatnego pola
        var pole = typeof(MojaKlasa).GetField("prywatnePole",
System.Reflection.BindingFlags.NonPublic | System.Reflection.BindingFlags.Instance);
        if (pole != null)
        {
            pole.SetValue(obj, nowaWartosc);
        }
    }

    public static void WywołajPrywatnaMetode(this MojaKlasa obj)
    {
        // Użycie refleksji do wywołania prywatnej metody
        var metoda = typeof(MojaKlasa).GetMethod("PrywatnaMetoda",
System.Reflection.BindingFlags.NonPublic | System.Reflection.BindingFlags.Instance);
        if (metoda != null)
        {
            metoda.Invoke(obj, null);
        }
    }
}

class Program
{
    static void Main(string[] args)
    {
        MojaKlasa mojObiekt = new MojaKlasa();
        mojObiekt.PokazPrywatnePole();

        // Użycie metody rozszerzającej do modyfikacji prywatnego pola
        mojObiekt.ZmienPrywatnePole(100);
        mojObiekt.PokazPrywatnePole();

        // Użycie metody rozszerzającej do wywołania prywatnej metody
        mojObiekt.WywołajPrywatnaMetode();

        Console.ReadLine();
    }
}

```

Metody do obsługi elementów statycznych klasy

Elementy statyczne klasy (ang. static members) to pola, właściwości i metody, które należą do klasy, a nie do jej instancji. Oznacza to, że są współdzielone przez wszystkie instancje klasy i mogą być dostępne bez tworzenia obiektu klasy.

Przykład 2: Metody do obsługi elementów statycznych klasy

```

using System;

namespace PrzykładElementyStatyczne
{
    public class MojaKlasa
    {
        // Statyczne pole
        public static int StatycznePole;

        // Statyczna właściwość
        public static int StatycznaWlasciowosc { get; set; }

        // Statyczny konstruktor
        static MojaKlasa()
        {
            StatycznePole = 5;
            StatycznaWlasciowosc = 10;
        }
    }
}

```

```

    }

    // Statyczna metoda
    public static void StatycznaMetoda()
    {
        Console.WriteLine("Wywołano statyczną metodę");
    }

    // Metoda instancji
    public void MetodaInstancji()
    {
        StatycznePole += 10;
        StatycznaWlasciwosc += 20;
        Console.WriteLine("Zmieniono wartości statycznych członków klasy");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Statyczne Pole: " + MojaKlasa.StatycznePole);
        Console.WriteLine("Statyczna Własność: " + MojaKlasa.StatycznaWlasciwosc);

        MojaKlasa.StatycznaMetoda();

        MojaKlasa mojObiekt = new MojaKlasa();
        mojObiekt.MetodaInstancji();

        Console.WriteLine("Statyczne Pole: " + MojaKlasa.StatycznePole);
        Console.WriteLine("Statyczna Własność: " + MojaKlasa.StatycznaWlasciwosc);

        Console.ReadLine();
    }
}

```

Zadania do samodzielnego rozwiązania

Zadanie 1. Dodaj do klasy MojaKlasa z przykładu funkcji zaprzyjaźnioną metodę, która będzie używać refleksji do wywołania prywatnej metody klasy. Następnie napisz metodę rozszerzającą, która będzie wywoływać tę prywatną metodę.

Zadanie 2. Rozbuduj przykład z elementami statycznymi klasy, dodając statyczny konstruktor, który będzie inicjalizował statyczne pole i właściwość. Dodaj również metodę instancji, która będzie modyfikować wartości statycznych członków klasy.

Zadanie 3. Napisz program, który będzie używał metod rozszerzających do dodania nowych funkcji do klasy MojaKlasa, takich jak logowanie zmian wartości pól prywatnych. Użyj refleksji, aby uzyskać dostęp do prywatnych członków klasy.